Generating a Minimum-Clue Sudoku with Logic-Based Benders Decomposition

Technical model note and implementation guide

September 30, 2026

Abstract

This note develops a mixed-integer programming model that generates both a completed Sudoku and the clues shown to a player. The objective is to minimize the number of clues while requiring that the displayed puzzle have exactly one completion. Uniqueness is enforced by a logic-based Benders decomposition: the master problem proposes a target Sudoku and a clue set, while a subproblem searches for a different valid Sudoku satisfying those clues. Whenever such an alternative is found, a conditional Benders cut is added. The cut does not ban the alternative Sudoku from ever becoming a target; instead it says that if the target is different from that Sudoku, at least one revealed clue must rule it out. This distinction is essential when the target Sudoku itself is a decision variable.

1 Problem statement

We want the optimization model to do two things simultaneously:

1. generate a valid completed Sudoku, which will be the intended solution; and
2. select as few entries of that solution as possible to reveal as clues.

The revealed clues must determine the generated solution uniquely.

For an $N \times N$ Sudoku, let

$$I = J = K = \{1, \dots, N\},$$

where i is a row, j is a column, and k is a digit. For the standard puzzle $N = 9$ and the boxes are 3×3; for the small test problem $N = 4$ and the boxes are 2×2.

Let $\mathcal{B}$ denote the set of boxes and let $B \subseteq I \times J$ denote the cells in a box.

2 Master variables

The master problem contains two main binary variable families:

$$y_{ijk} = \begin{cases} 1, & \text{if the generated target Sudoku places digit } k \text{ in cell } (i,j), \\ 0, & \text{otherwise,} \end{cases}$$

and

$$x_{ijk} = \begin{cases} 1, & \text{if digit } k \text{ is actually shown as a clue in cell } (i,j), \\ 0, & \text{otherwise.} \end{cases}$$

The objective is simply

$$\min \sum_{i \in I} \sum_{j \in J} \sum_{k \in K} x_{ijk}. \tag{1}$$

Plain-language meaning. Every selected x is one visible clue, so the objective asks the model to make the puzzle as sparse as possible.

3 Sudoku constraints in the master

3.1 One digit in every cell

$$\sum_{k \in K} y_{ijk} = 1 \qquad \forall i \in I,\ j \in J. \tag{2}$$

Plain-language meaning. Every square of the completed answer key must contain exactly one number.

3.2 Each digit once in every row

$$\sum_{j \in J} y_{ijk} = 1 \qquad \forall i \in I,\ k \in K. \tag{3}$$

Plain-language meaning. In each row, every digit appears exactly once.

3.3 Each digit once in every column

$$\sum_{i \in I} y_{ijk} = 1 \qquad \forall j \in J,\ k \in K. \tag{4}$$

Plain-language meaning. In each column, every digit appears exactly once.

3.4 Each digit once in every box

$$\sum_{(i,j) \in B} y_{ijk} = 1 \qquad \forall B \in \mathcal{B},\ k \in K. \tag{5}$$

Plain-language meaning. Every sub-box also contains each digit exactly once.

3.5 A clue must agree with the generated target

$$x_{ijk} \le y_{ijk} \qquad \forall i, j, k. \tag{6}$$

Plain-language meaning. The model is not allowed to print a clue that disagrees with its own answer key. If the generated answer has a 7 in a cell, the only clue that can be shown in that cell is 7.

Because equation (2) permits only one target digit per cell, equation (6) also implies that at most one clue can be selected in each cell.

4 Why uniqueness is the difficult part

Constraints (2)–(6) generate a valid answer key and a subset of its entries. They do *not* guarantee that a player sees enough information to recover that answer uniquely.

Given a candidate pair $(\bar{x}, \bar{y})$, uniqueness means:

> There must not exist another complete Sudoku $z \neq \bar{y}$ that agrees with every clue in $\bar{x}$.

Writing that requirement directly in a compact MIP is awkward because it is a statement about the *nonexistence* of another integer solution. This is the role of the Benders subproblem.

5 The uniqueness subproblem

For a fixed master solution $(\bar{x}, \bar{y})$, introduce binary variables

$$z_{ijk} = 1 \quad \Longleftrightarrow \quad \text{the candidate alternative solution puts digit } k \text{ in cell } (i, j).$$

The z variables satisfy the same four families of Sudoku constraints as y.

5.1 The alternative must respect all selected clues

$$z_{ijk} \ge \bar{x}_{ijk} \qquad \forall i, j, k. \tag{7}$$

Plain-language meaning. If the master printed a digit as a clue, the alternative solution must contain that digit in that cell. Blank cells remain unrestricted.

Let $\bar{k}_{ij}$ be the digit selected by the current target $\bar{y}$ in cell (i, j). To stop the subproblem from simply returning the target itself, impose

$$\sum_{i \in I} \sum_{j \in J} z_{ij,\bar{k}_{ij}} \le N^2 - 1. \tag{8}$$

Plain-language meaning. A complete Sudoku agrees with the target in all N^2 cells only when it *is* the target. Equation (8) forces the subproblem to disagree in at least one cell.

The basic subproblem can therefore have the feasibility objective

$$\min 0.$$

If it is infeasible, no second completion exists and the puzzle is unique.

A useful strengthening is to search for the alternative closest to the current target:

$$\max \sum_{i,j} z_{ij,\bar{k}_{ij}}. \tag{9}$$

This minimizes Hamming distance from the target. Such alternatives tend to produce stronger cuts, although the optimization can be substantially more expensive for 9×9 Sudoku.

6 The essential endogenous-target Benders cut

Suppose the subproblem finds a valid alternative Sudoku s. If the target had been fixed in advance, we could simply require at least one clue in a cell where the target differs from s. Here, however, the master is allowed to change the target on its next solve. Therefore a cut tied permanently to the old target would not be globally valid.

The correct cut is conditional.

For every discovered complete Sudoku s, introduce a variable

$$0 \le \delta_s \le 1.$$

Use the linking constraints

$$\delta_s + y_{ij,s_{ij}} \ge 1 \qquad \forall i, j. \tag{10}$$

If the current target equals s, then $y_{ij,s_{ij}} = 1$ in every cell and δ_s may be zero. If the target differs from s in even one cell, then the corresponding $y_{ij,s_{ij}} = 0$, forcing $\delta_s = 1$.

Now add the cut

$$\sum_{i \in I} \sum_{j \in J} \sum_{\substack{k \in K \\ k \ne s_{ij}}} x_{ijk} \ge \delta_s. \tag{11}$$

Plain-language meaning. There are two possibilities:

- If the master decides that s itself will be the official answer key, the cut switches off because δ_s can be zero.
- If the official answer key is anything other than s, then $\delta_s = 1$, and at least one displayed clue must be a digit that s does not contain in that cell. Therefore s can no longer solve the displayed puzzle.

This is the key point when the target is endogenous: *we do not globally ban a valid Sudoku s*. It is perfectly acceptable for s to become the target. We ban s only from being a second solution to a puzzle whose target is different.

7 Exactly how a non-unique incumbent is eliminated

Assume the current master proposes $(\bar{x}, \bar{y})$ and the subproblem finds a second completion $s \neq \bar{y}$.

Because s satisfies all current clues, every currently selected clue agrees with s. Therefore, at the incumbent,

$$\sum_{i,j} \sum_{k \neq s_{ij}} \bar{x}_{ijk} = 0.$$

At the same time, $\bar{y} \neq s$, so equation (10) forces

$$\delta_s = 1.$$

Hence the new cut requires

$$0 \geq 1,$$

which is false. The current master solution is therefore cut off.

On the next master solve, the optimizer has three conceptual choices:

1. keep essentially the same target and add at least one clue that s violates;
2. change the target to s, in which case s is no longer an alternative and the cut can switch off; or
3. change to some other target, but still select at least one clue incompatible with s.

The procedure repeats. Each successful subproblem contributes another valid complete Sudoku that the master must either adopt as its target or explicitly rule out with a clue.

8 Logic-based Benders algorithm

A textbook version of the algorithm is:

1. Start with no Benders cuts.
2. Solve the restricted master:
$$\min \sum_{ijk} x_{ijk}$$
subject to the Sudoku constraints on y, the clue-link constraints, and all cuts generated so far.
3. Let $(\bar{x}, \bar{y})$ be the master optimum.
4. Solve the uniqueness subproblem: find a Sudoku z satisfying $\bar{x}$ with $z \neq \bar{y}$.
5. If the subproblem is infeasible, stop. The puzzle is unique.
6. Otherwise let s be the alternative Sudoku found, add equations (10) and (11), and return to Step 2.

Several alternatives can be generated before returning to the master. To do this, add a temporary no-good constraint for each alternative already found in the batch:

$$\sum_{i,j} z_{ij,s_{ij}} \leq N^2 - 1. \tag{12}$$

This produces a multi-cut implementation.

9 Why the final solution is not only unique but optimal

The restricted master contains only a subset of all uniqueness cuts, so its objective value is a lower bound on the true minimum number of clues.

Suppose the restricted master returns a puzzle with q clues and the uniqueness subproblem is infeasible. Then:

- the current puzzle is feasible for the full problem because it has no second completion;
- therefore the true optimum is at most q;
- the restricted-master optimum is a lower bound, so the true optimum is at least q.

Consequently the true optimum equals q.

This gives an exact optimality certificate, provided each restricted master is solved to optimality and the final uniqueness subproblem is proved infeasible.

10 A layman's analogy

The algorithm can be viewed as a repeated game between a *puzzle designer* and a *checker*.

The designer writes an answer key and reveals as few numbers as possible. The checker then tries to "cheat" by finding a different completed Sudoku that still matches all visible clues.

If the checker succeeds with a grid s, the designer receives a new rule:

> Unless you decide that s is your official answer key, you must reveal at least one clue that makes s impossible.

The designer then redesigns the puzzle under all accumulated rules. When the checker can no longer find a second completion, the current puzzle is unique.

11 Relationship to classical Benders decomposition

Classical Benders decomposition often starts from a problem of the form

$$\min_{x \in X} \; c^T x + Q(x), \tag{13}$$

where x are master variables and $Q(x)$ is the value of a second-stage linear optimization problem. In two-stage stochastic programming,

$$Q(x) = \sum_{\omega \in \Omega} p_\omega Q_\omega(x), \tag{14}$$

with one recourse problem for each random scenario ω.

When the recourse problem is linear, LP duality generates Benders cuts. A typical optimality cut has the form

$$\theta_\omega \geq \alpha_\omega + \beta_\omega^T x,$$

where the coefficients come from an optimal dual solution. Feasibility cuts may similarly come from dual extreme rays.

The Sudoku decomposition is different in several important ways.

	Classical stochastic Benders	**Sudoku logic-based Benders**
Meaning of subproblem	Evaluate recourse cost or feasibility after first-stage decisions	Search for a violating second Sudoku completion
Uncertainty	Explicit random scenarios with probabilities	No randomness; alternatives are adversarial witnesses
Subproblem type	Commonly LP recourse	Integer feasibility/MIP
How cuts are obtained	LP dual solutions or rays	Logical implication from a discovered complete Sudoku
Master auxiliary value	Often θ approximating recourse value	No recourse-value variable is needed
Meaning of a cut	Improve approximation of $Q(x)$ or restore recourse feasibility	Require a clue that eliminates a known alternative unless it becomes the target
Closest conceptual relatives	L-shaped method, stochastic programming	Logic-based Benders, separation, row generation, branch-and-cut

Thus it is reasonable to call the method Benders decomposition because a master proposal is tested by a subordinate optimization problem and violations are returned as master cuts. More precisely, however, it is **logic-based Benders decomposition** or **constraint generation**, rather than the classical LP-duality Benders method.

12 Why simplex pivots are not a uniqueness test

An LP view can still provide intuition, but Sudoku uniqueness is an integer property. The LP relaxation can contain fractional points even when there is only one integer completion. Conversely, a unique optimum for an arbitrary LP objective does not imply that only one feasible integer Sudoku exists; another valid Sudoku may simply have a worse objective value.

For that reason, "no zero-cost pivot" or "unique LP basis" is not the right certificate. The exact certificate is instead:

> The integer subproblem searching for a distinct completion is infeasible.

That statement is precisely the definition of uniqueness for the displayed clue set.

13 Symmetry breaking

When the target Sudoku is generated by the master, many mathematically equivalent answer keys exist because the digits can be relabelled. A safe symmetry-breaking condition is

$$y_{1,j,j} = 1 \qquad j = 1, \ldots, N, \tag{15}$$

which fixes the first row to $1, 2, \ldots, N$.

This does not remove any essential puzzle structure: any valid Sudoku has all digits exactly once in its first row, so its symbols can be relabelled to obtain this canonical form. Symmetry breaking is much more important in the endogenous-target version than in the fixed-target version because the optimizer otherwise explores many equivalent answer keys.

13.1 Ordering rows and bands

Digit relabelling is only one part of the Sudoku symmetry group. Permuting the rows inside a band, or permuting whole bands, maps a valid grid to a valid grid and a unique puzzle to a unique puzzle with the same number of clues. Permutations that keep row 1 in place also keep the first row equal to $1, \ldots, N$, so they can be combined with the constraint above. Write

$$v_i = \sum_{k \in K} k \, y_{i1k}$$

for the digit in column 1 of row i, and let $b = \sqrt{N}$. For consecutive rows $i < i'$ of the same band, both different from row 1,

$$v_i + 1 \le v_{i'}, \tag{16}$$

and for the first rows of consecutive bands $2, \ldots, b$,

$$v_{(\beta-1)b+1} + 1 \le v_{\beta b+1} \qquad \beta = 2, \ldots, b-1. \tag{17}$$

Any grid can be brought into this form by first sorting the rows inside each band and then sorting bands $2, \ldots, b$, so no equivalence class is lost. The full symmetry group is much larger (3,359,232 grid transformations times 9! relabellings for 9×9, leaving about 5.47×10^9 essentially different grids [7]).

13.2 Splitting by the top band

A stronger reduction fixes the whole top band. Two bands are called equivalent if one can be turned into the other by permuting stacks, permuting columns inside stacks, permuting the band's three rows and relabelling digits. All of these operations are Sudoku symmetries, and permuting bands can move any band of a grid to the top. Hence every grid is equivalent to a grid whose top band is one representative T of its band class.

The script `make_top_bands.py` enumerates all 2,612,736 bands whose first box is $1, \ldots, 9$ in reading order and splits them into orbits under the operations above. It finds 416 classes, in agreement with the literature [13]; for 4×4 there are 2. For class representative T,

$$y_{ijk} = [\, T_{ij} = k \,] \qquad i = 1, \ldots, b, \; j \in J, \; k \in K, \tag{18}$$

which replaces the first-row constraint. Rows inside bands $2, \dots, b$ and the order of those bands can still be normalised with (16)–(17). The minimum clue count over all grids is the minimum over the 416 class models, so a lower bound for the whole problem is the smallest lower bound over the classes.

14 Stronger Benders cuts from unavoidable sets

In Sudoku terminology, a set U of cells of a completed grid G is *unavoidable* if the entries of U can be rearranged to give a different valid grid [13]. Every unique puzzle for G must contain a clue in every unavoidable set, and conversely a clue set that hits every unavoidable set determines G uniquely.

Each alternative s found by the subproblem yields such a set for the current target $\bar{y}$:

$$D = \{(i, j) : s_{ij} \neq \bar{k}_{ij}\}.$$

Inside each row, column and box, $\bar{y}$ and s agree outside D, so the digits of $\bar{y}$ on D are a rearrangement of the digits of s on D. Consequently *any* valid grid that agrees with $\bar{y}$ on D stays valid when its entries on D are replaced by those of s. This gives a cut that is valid for every target, not only for $\bar{y}$:

$$\sum_{(i,j)\in D} (x_{ij\bar{k}_{ij}} - y_{ij\bar{k}_{ij}}) \geq 1 - |D|. \tag{19}$$

Plain-language meaning. If the answer key has the same digits as $\bar{y}$ on the cells D, then at least one of those cells must be shown as a clue; otherwise the digits on D could be swapped to those of s. If the target differs from $\bar{y}$ somewhere on D, the left-hand side is at least $-|D| + 1$ and the cut is inactive.

At the incumbent all $|D|$ target terms equal one and no clue lies in D, so the left-hand side is $-|D|$ and the incumbent is cut off. Compared with (11), the cut needs no auxiliary variable and has only $2|D|$ nonzeros. It also applies to every target that shares the pattern on D, not only to targets different from s. Small sets D give the strongest cuts, which is a further argument for the closest-alternative objective (9).

15 Valid inequalities independent of the target

The following inequalities hold for every unique puzzle of every grid, so they can be added to the master before the first solve.

15.1 At least $N - 1$ digits appear among the clues

If two digits $a \neq b$ both have no clue, exchanging a and b everywhere in the target gives a different valid grid that agrees with all clues. Hence at most one digit may be missing [13]. With binary u_k,

$$u_k \leq \sum_{i,j} x_{ijk} \quad \forall k, \qquad \sum_{k\in K} u_k \geq N - 1. \tag{20}$$

Since $\sum_{ijk} x_{ijk} \geq \sum_k u_k$, the LP bound of the master immediately rises to $N - 1$, i.e. 8 for 9×9, instead of 0.

15.2 No two empty rows in a band

Two rows of the same band can be exchanged without violating any Sudoku constraint. If both rows contain no clue, the exchanged grid agrees with every clue and differs from the target. Therefore

$$\sum_{j,k}(x_{ijk} + x_{i'jk}) \geq 1 \qquad \text{for rows } i < i' \text{ in the same band,} \tag{21}$$

and the same for two columns of the same stack.

15.3 Rectangles: unavoidable sets of four cells

Let rows $i < i'$ and columns $j < j'$ be such that the rows share a band or the columns share a stack, and let $a \neq b$ be digits. If the target has a, b in $(i, j), (i, j')$ and b, a in $(i', j), (i', j')$, exchanging a and b in these four cells gives another valid grid: every row and column still contains a and b once, and each affected box loses and gains the same digit. With $R = \{(i, j, a), (i, j', b), (i', j, b), (i', j', a)\}$,

$$\sum_{(p,q,k)\in R} x_{pqk} \;\geq\; \sum_{(p,q,k)\in R} y_{pqk} - 3. \tag{22}$$

For 9×9 there are 40,824 such inequalities with eight nonzeros each. They are best placed in the solver's lazy-constraint pool, so they only enter the LP when violated.

16 Faster restricted-master solves

In the iterative algorithm each master is solved from scratch to proven optimality, and late in the run these solves dominate the time. Three simple measures help, none of which changes the result.

1. **A lower-bound row.** Let L be the optimal value of the last restricted master. Later masters only have more cuts, so their optimum is at least L, and the row

   $$\sum_{ijk} x_{ijk} \geq L$$

   is valid. With it the solver can stop as soon as it finds a solution with L clues instead of proving the bound again.

2. **A repaired start.** The previous master solution violates the new cuts, but it becomes feasible again after adding a clue, at the target value, in each set D that it does not yet hit. Adding clues never violates an earlier cut or any of the inequalities of the previous section, so the result is a valid MIP start. Cells are chosen greedily to hit as many sets at once as possible.

3. **Dropping dominated cuts.** Write the cut (19) with the key $\{(i, j, \bar{k}_{ij}) : (i, j) \in D\}$. If one key is contained in another, the cut with the smaller key implies the other, also for the LP relaxation, so the larger one is not added or is removed.

In practice none of the three paid off. On the fixed 9×9 grid of [13] with Gurobi 14 (300 s per run), the time to reach the bound 16 was 53 s without them, 58 s with the start, 59 s with the pruning, 80 s with the lower-bound row and 110 s with all three. The lower-bound row is the clearest loss: an explicit bound on the objective makes the LP relaxation highly degenerate. The implementation therefore keeps them only as options, switched off by default.

17 An assumed minimum and a positional tie-breaker

17.1 At least 17 clues

The computation of [13] shows that every 9×9 Sudoku with a unique completion has at least 17 clues. Taking this as known,

$$\sum_{ijk} x_{ijk} \geq 17 \tag{23}$$

may be added to the master. The objective still minimises the number of clues, so if no unique 17-clue puzzle is found within the time limit, the best unique puzzle found so far, possibly with more clues, is reported. The constraint removes the work of raising the lower bound from 8 to 17, but not the search for a grid that actually admits a 17-clue puzzle, and such grids are rare [13]. Unlike the inequalities of the previous sections, (23) is not proved by the model; the reported lower bound then rests on that result.

17.2 Preferring clues towards the top-left

Many puzzles have the same minimum number of clues. A secondary preference for clues near the top-left corner, with weight $2 + i + j$ for cell (i, j) counted from zero, can be combined with the clue count lexicographically:

$$\min \sum_{ijk} (W + 2 + i + j)\, x_{ijk}, \qquad W = 2N^3 + 1. \tag{24}$$

At most one clue lies in each cell and each weight is below $2N$, so the secondary part is always smaller than W. Therefore the clue count is minimised first and the positional weights only choose among puzzles with the same count, and $\lfloor z/W \rfloor$ of any objective bound z is a lower bound on the clue count. This selects a single preferred puzzle among equivalent ones, but it does not remove symmetric solutions from the feasible set as the symmetry-breaking constraints do. Weighting the digit k as well would add nothing, because the digit relabelling is already fixed by the first-row or top-band constraints.

18 Controlling the spread of the clues

Following Ateş and Çavdur [1], the number of clues in each row, column and box, and the number of times each digit is shown, can be bounded. In our variables these are linear:

$$\ell_R \leq \sum_{j,k} x_{ijk} \leq u_R \quad \forall i, \qquad \ell_C \leq \sum_{i,k} x_{ijk} \leq u_C \quad \forall j, \tag{25}$$

$$\ell_B \leq \sum_{(i,j)\in B} \sum_{k} x_{ijk} \leq u_B \quad \forall B \in \mathcal{B}, \qquad \ell_K \leq \sum_{i,j} x_{ijk} \leq u_K \quad \forall k.$$

In [1] the digit bound needs a product of a grid variable and a "shown" variable, which is then linearised; here x_{ijk} already records both the cell and the digit of a clue, so no extra variables are needed.

Unlike the inequalities of Section 16 and before, (25) are *requirements*: they exclude puzzles that are unique but unbalanced, so the optimum becomes the fewest clues among balanced unique puzzles.

Because the same bounds apply to every row, column, box and digit, they are unchanged by all the symmetries used above, and the symmetry-breaking constraints remain valid. If the bounds admit no unique puzzle, the master eventually becomes infeasible, which the implementation reports as such. The inequality (20) is the special case $\ell_K = 1$ for all but one digit.

19 Fixed-target variant

If the target grid is given, only the clue variables remain and the problem becomes the minimum-clue problem for one grid, as in the bilevel formulation of [3]. The conditional cut (11) then coincides with the unavoidable-set cut (19), and symmetry breaking must be switched off because it would contradict the given grid. The 17-clue puzzle of [13] gives a grid whose optimum is known to be exactly 17, which makes a useful 9×9 benchmark.

20 The 4×4 test problem

The implementation included with this note solves a 4×4 instance using SciPy's `milp` interface, whose MIP backend is HiGHS. One run generates the target

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \\ 3 & 1 & 4 & 2 \\ 2 & 4 & 1 & 3 \end{bmatrix}$$

and the four-clue puzzle

$$\begin{bmatrix} 1 & 2 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & 4 & \cdot \\ 2 & \cdot & \cdot & \cdot \end{bmatrix}.$$

The final subproblem is infeasible, proving that this displayed puzzle has no second completion. Because the restricted master objective is four at that point, the same calculation also proves that four clues are optimal for the model under the stated symmetry normalization.

21 Command-line and Colab use

The command-line implementation can be run as

```
python sudoku_benders_generator.py --size 4
```

For a 9×9 run, a practical starting point is

```
python sudoku_benders_generator.py --size 9 \
    --separator feasible --batch-size 4 \
    --max-master-solves 500
```

The `closest` separator generally gives stronger cuts but can be considerably more expensive on 9×9. The `feasible` separator asks HiGHS only to find some second completion.

The Colab notebook contains the same model but exposes the settings as ordinary Python variables rather than command-line arguments.

Gurobi version

`sudoku_benders_gurobi.py` implements the same model with `gurobipy`, together with the extensions of the previous sections. It offers two solution modes. In `callback` mode a single branch-and-cut run separates every incumbent in a `MIPSOL` callback and adds the cuts as lazy constraints. In `iterative` mode a persistent master is re-optimised after each round of cuts, as in the algorithm above. The main options are:

Option	Meaning
`-cut delta|unavoidable|both`	(10)–(11), (19), or both
`-digit-cover`	inequalities (20)
`-line-pairs`	inequalities (21) for rows and columns
`-rectangles lazy|model`	inequalities (22)
`-strengthen`	all three of the above (rectangles lazy)
`-row-symmetry`	ordering constraints (16)–(17)
`-fixed-grid GRID`	fixed-target variant, e.g. `mcguire17`
`-top-band I|I-J|all`	fix the top band (18), one model per class
`-speedups LIST`	measures of Section 16: `bound,start,prune` or `none`
`-min-clues M`	assumed minimum (23), e.g. 17
`-position-weights`	positional tie-breaker (24)
`-row-clues`, `-col-clues`, `-box-clues`, `-digit-clues`	bounds (25) as `MIN:MAX`, `MIN:`, `:MAX` or an exact number

For example:

```
python sudoku_benders_gurobi.py --size 9 --fixed-grid mcguire17 \
    --mode iterative --strengthen --cut unavoidable \
    --separator closest --batch-size 20 --max-master-solves 100000
```

Every returned puzzle is re-checked for uniqueness at the end, and a result that fails the check is never reported as optimal.

22 Computational comments for 9×9

The mathematical formulation does not change when moving from 4×4 to 9×9, but the computational problem changes dramatically. The master is simultaneously searching over possible target grids and clue sets, while each Benders iteration solves another integer Sudoku search problem. A complete proof of minimum clue count can therefore require many cuts and many master re-solves.

For larger instances, useful enhancements include stronger symmetry breaking, multi-cut separation, cut-pool management, warm-start clue sets, and a hybrid separator that normally seeks any feasible

alternative but periodically searches for a closest alternative. These are performance improvements; they do not change the logic of the uniqueness certificate.

23 Related work

Integer programming for Sudoku. Solving Sudoku as a binary program with the four assignment constraint families is a classical teaching example [4, 11, 2]. Bartlett et al. [2] also derive new puzzles from a given one through the symmetry operations of Section A, and Williams [18] certifies uniqueness with a second integer program, which is our subproblem without the cut-generation loop.

Generating Sudoku puzzles with a MIP. The closest work is that of Ateş and Çavdur [1]. Like our master, their model chooses both the solution grid and the shown entries and minimises the number of shown entries, and they note that earlier studies had not combined both phases in one mathematical program. Their main addition is the control of the spread of clues and digit frequencies, adopted in Section 18. Their model does *not* enforce uniqueness; the authors state this as a limitation and list it as future work. Uniqueness is instead obtained outside the model, either by re-solving until an enumeration finds no second completion, or, in their game application, by a backtracking procedure that builds a random grid and removes entries while the solution stays unique. Their model also discusses reduced costs of the LP relaxation in connection with alternative solutions; as argued in Section 12, such tests do not certify integer uniqueness. The present note adds the missing piece: uniqueness is enforced inside the optimisation through logic-based Benders cuts, which also yields lower bounds on the number of clues.

The minimum number of clues. McGuire, Tugemann and Civario [13] proved by exhaustive hitting-set enumeration over all essentially different grids [15] that 17 clues are necessary (Appendix A); Lin and Wu [12] described a related search. Tjusila, Besançon, Turner and Koch [3] formulate the minimum-clue problem for a *fixed* grid as a binary bilevel program strengthened by unavoidable-set inequalities, and solve 95% of 50 test grids. Our fixed-target variant addresses the same problem by Benders decomposition, while the endogenous-target model also chooses the grid.

Other puzzles. Mixed-integer models that generate as well as solve puzzles exist for Smashed Sums [10] and the Survo puzzle [17], and Hartmann [5, 6] solves several smartphone puzzles by mathematical programming. For Evolomino, Nikolaev and Myasnikov [14] build a random solution and then remove clues in random order, keeping a removal only if the ILP with a no-good constraint excluding the known solution becomes infeasible; the result is minimal (no clue can be removed) but not necessarily minimum. The same no-good test is used to check uniqueness of KenKen solutions [9]. In constraint programming, Simonis [16] models Kakuro, compares MILP and SAT solvers and tightens puzzles by removing hints, and Jefferson, Moncur and Petrie [8] generate, check and grade all levels of a commercial puzzle game with constraint models.

Summary of the difference. Published generators either ignore uniqueness in the optimisation model [1] or use the solver only as a uniqueness oracle inside a greedy removal loop [14, 16]. The latter is fast but gives no guarantee on the number of clues. Our decomposition optimises over

grids and clue sets jointly with uniqueness enforced by cuts, at a much higher computational cost; a greedy removal pass would be a natural way to provide it with good starting solutions.

24 Summary

The full method can be summarized in one sentence:

> Generate a Sudoku and a small set of its entries, then repeatedly ask an integer subproblem to find another completion; every successful alternative becomes a conditional master cut requiring a clue that rules it out unless that alternative becomes the new target.

The central cut is

$$\delta_s + y_{ij,s_{ij}} \geq 1 \quad \forall i,j, \qquad \sum_{i,j,k \neq s_{ij}} x_{ijk} \geq \delta_s.$$

It is this conditional structure that makes the decomposition valid when the target Sudoku is itself generated by the MIP.

A McGuire's proof in plain language

This appendix summarises, without technical detail, how McGuire, Tugemann and Civario [13] settled the minimum-clue question, and how their approach relates to the model in this note.

The question

A newspaper Sudoku usually shows about 25 clues. Puzzles with 17 clues were known, but nobody had ever found one with 16. The question was whether a 16-clue puzzle with exactly one solution exists. If none exists, 17 is the minimum.

Not an optimisation model

The proof did not use mixed-integer programming or any general-purpose solver. The authors wrote a specialised program, called *checker*, in C++ with its innermost routines in assembly language. It answers one yes/no question for one completed grid: is there a set of 16 of its entries that determines the grid uniquely? They then ran it on every essentially different completed grid. So instead of asking "what is the fewest clues?" for all grids at once, as our master problem does, they asked "are 16 enough?" for each grid separately.

Step 1: a catalogue of all grids

There are about 6.7×10^{21} completed Sudoku grids, far too many to check. But relabelling the digits, swapping bands or stacks, reordering rows inside a band or columns inside a stack, and transposing (reflecting) the grid all turn one grid into an equivalent one. If one grid has a 16-clue

puzzle, so do all its equivalent grids. Earlier work by Russell and Jarvis had shown that there are 5,472,730,538 essentially different grids. A program by Glenn Fowler listed one representative of each, compressed to under 6 GB, so that the whole catalogue fitted on a single DVD.

Step 2: unavoidable sets

In a completed grid, some groups of cells can be rearranged to give a different valid grid. The simplest example is a "rectangle" of four cells holding two digits, say

$$\begin{array}{cc} 5 & 9 \\ 9 & 5 \end{array}$$

in two rows and two columns arranged so that swapping the 5s and 9s breaks no rule. If a puzzle shows none of these four cells, the solver cannot tell the two versions apart. So every valid puzzle must reveal at least one cell of every such group. The authors call these groups *unavoidable sets*; in the study of Latin squares they are known as *trades*. Our cuts (19) and (22) are exactly this idea.

For each grid, *checker* looked up a list of 525 known patterns ("blueprints", compiled by Ed Russell) of up to twelve cells and matched them against the grid. A typical grid has about 360 unavoidable sets of this size, which took under a twentieth of a second to find.

Step 3: choosing clues that hit every set

A candidate puzzle must contain a cell from every unavoidable set. Choosing 16 cells that together touch all the sets is the classical *hitting set* problem. Instead of trying all $\binom{81}{16} \approx 3.4 \times 10^{16}$ choices of 16 cells, *checker* built candidates one clue at a time. It always took the next clue from an unavoidable set that no clue had touched yet. That set must be hit anyway, so only its cells need to be tried.

The decisive speed-up was early abandonment. If several unavoidable sets share no cells, each needs its own clue. For example, if after 13 clues there are still four untouched and mutually disjoint sets, three more clues cannot possibly be enough, and that branch of the search can be dropped immediately. The authors call such groups of disjoint sets "cliques". They report that this simple observation was what made the project feasible. A second trick, a "dead clue" list, ensured that no candidate was generated twice.

Step 4: testing each candidate

A set of 16 clues that hits all the listed unavoidable sets is only a candidate, because the list contains only the sets of up to twelve cells. Each candidate was therefore given to a fast conventional Sudoku solver (a stripped-down version of Brian Turner's *BBSudoku*), which checked more than 50,000 puzzles per second for a second solution. No candidate ever had exactly one solution.

The computation

Step 3 used about 95% of the running time. The search ran from January to December 2011 on the *Stokes* cluster of the Irish Centre for High-End Computing and took about 7.1 million core hours,

with an average of 3.6 seconds per grid. The authors announced on 1 January 2012 that no 16-clue Sudoku exists, so the minimum is 17.

To check that *checker* was correct, they also ran a version that searched for 17-clue puzzles on all grids already known to contain them. It found every known 17-clue puzzle on those grids. They also verified each unavoidable set found and each solver answer a second time.

Where the 17-clue puzzles came from

The 17-clue puzzle quoted in the paper, which our `mcguire17` grid is based on, was not found by this computation. Seventeen-clue puzzles had been collected for years before, mostly by Gordon Royle while searching for a 16-clue puzzle, with contributions from the online Sudoku community. His list had grown to 49,151 puzzles. Contributors rarely sent in new puzzles by then, which suggested the list was nearly complete. One grid holds the record with 29 different 17-clue puzzles, all found by Royle; the authors later searched this grid completely and confirmed that there are exactly 29. So the existence of 17-clue puzzles was known from examples, and the contribution of [13] was to prove that 16 is never enough.

What this means for our model

Both approaches rest on unavoidable sets, but they use them differently:

- *checker* works on one fixed grid at a time, as in our fixed-target variant. It finds its unavoidable sets in advance from the blueprint list, whereas our subproblem discovers them one by one as it finds alternative solutions.
- *checker* enumerates clue sets with a highly tuned search that stops a branch as soon as the untouched disjoint sets show it cannot succeed. Our master leaves that reasoning to a general MIP solver working from the LP relaxation.
- Symmetry is handled completely by the catalogue of 5.47×10^9 representatives. Our model breaks only part of the symmetry, through the first row or the top-band classes.

The comparison also shows the scale: *checker* needed 3.6 seconds on average to decide whether one grid has a 16-clue puzzle, while our fixed-grid run needed about 11 minutes to raise the lower bound to 17 for a single grid.

References

[1] T. Ateş, F. Çavdur. Sudoku puzzle generation using mathematical programming and heuristics: puzzle construction and game development. *Expert Systems with Applications* 282 (2025) 127710. doi:10.1016/j.eswa.2025.127710.

[2] A. C. Bartlett, T. P. Chartier, A. N. Langville, T. D. Rankin. An integer programming model for the Sudoku problem. *Journal of Online Mathematics and its Applications* 8(1), 2008.

[3] G. Tjusila, M. Besançon, M. Turner, T. Koch. How many clues to give? A bilevel formulation for the minimum Sudoku clue problem. *Operations Research Letters*, 2024. arXiv:2305.01697.

[4] M. J. Chlond. Classroom exercises in IP modeling: Su Doku and the log pile. *INFORMS Transactions on Education* 5(2) (2005) 77–79.

[5] S. Hartmann. Puzzle-solving smartphone puzzle apps by mathematical programming. *INFORMS Transactions on Education* 18(2) (2018) 127–141.

[6] S. Hartmann. Puzzle—more logic puzzle apps solved by mathematical programming. *INFORMS Transactions on Education* 20(1) (2019) 49–55.

[7] A. F. Jarvis. Sudoku enumeration: the symmetry group. http://www.afjarvis.org.uk/sudoku/sudgroup.html.

[8] C. Jefferson, W. Moncur, K. E. Petrie. Combination: automated generation of puzzles with constraints. In *Proceedings of the ACM Symposium on Applied Computing (SAC '11)*, 2011, 907–912.

[9] E. Kalvelagen. Unique solutions in KenKen. *Yet Another Math Programming Consultant* (blog), December 2016. http://yetanothermathprogrammingconsultant.blogspot.com/2016/12/unique-solutions-in-kenken.html.

[10] B. Keçeci. A mixed integer programming formulation for Smashed Sums puzzle: generating and solving problem instances. *Entertainment Computing* 36 (2021) 100386.

[11] T. Koch. Rapid mathematical programming or how to solve Sudoku puzzles in a few seconds. In *Operations Research Proceedings 2005*, Springer, 2006, 21–26.

[12] H.-H. Lin, I-C. Wu. An efficient approach to solving the minimum Sudoku problem. *ICGA Journal* 34(4) (2011) 191–208.

[13] G. McGuire, B. Tugemann, G. Civario. There is no 16-clue Sudoku: solving the Sudoku minimum number of clues problem via hitting set enumeration. *Experimental Mathematics* 23(2) (2014) 190–217. arXiv:1201.0749.

[14] A. V. Nikolaev, Y. A. Myasnikov. An integer linear programming model for the Evolomino puzzle. arXiv:2603.09483, 2026.

[15] E. Russell, F. Jarvis. Mathematics of Sudoku II. *Mathematical Spectrum* 39(2) (2006) 54–58.

[16] H. Simonis. Kakuro as a constraint problem. In *Proceedings of the ModRef Workshop at CP*, 2008.

[17] B. Sungur, F. S. Madenoğlu. An integer programming formulation for generating and solving Survo puzzle. *Erciyes University Journal of Faculty of Economics and Administrative Sciences*, 2025.

[18] A. Williams. When natural variables are not enough: teaching integer programming with Sudoku. arXiv:2606.30227, 2026.